Canadian International College
Faculty of Business Information Technology

Stock Market Analysis System

Project Documentation

Supervised by
Dr. Khalaf

Prepared by

Name	Student ID	Role
Shahd Maher	202207065	Member 1 – Project Lead / Backend
Zeina Sherif	202206582	Member 2 – Data Processing & Analysis
Rama said	202206960	Member 3 – Visualization Developer
Mayar Sherief	202206131	Member 4 – UI Developer / Tester / Docs
Rayan Hesham	202206466	Member 5 – Supporting Developer

May 2026

1. PROJECT OVERVIEW

This project is a web-based application developed using Streamlit that allows users to view and analyze stock market data. The system provides simple visualizations and basic stock information to help users understand market trends.

2. OBJECTIVES

- To understand how stock market data is collected and displayed
- To build a simple interactive web application
- To visualize stock price trends using charts
- To provide basic stock information to users

3. SCOPE

The system will:

- Allow users to search for stock symbols
- Display current and historical stock prices
- Show graphical representation of stock trends
- Provide a simple and user-friendly interface

4. FUNCTIONAL REQUIREMENTS

The application should:

- Accept stock symbol input from the user
- Fetch stock data from an API (e.g., Yahoo Finance)
- Display:
  - Stock name and current price
  - Historical data (last 7 days / 1 month)
- Generate line charts for price trends
- Handle invalid inputs with error messages

5. NON-FUNCTIONAL REQUIREMENTS

- The system should be easy to use
- The interface should be simple and clear

- Data should load quickly
- The application should be reliable and responsive

6. TOOLS AND TECHNOLOGIES

- Frontend & Backend: Streamlit
- Programming Language: Python
- Libraries:
  - Pandas (data handling)
  - Matplotlib or Plotly (visualization)
  - yfinance (stock data API)

7. SYSTEM WORKFLOW

1. User enters a stock symbol (e.g., AAPL)
2. The system retrieves stock data from the API
3. The data is processed and displayed
4. Charts are generated to show trends

8. USER INTERFACE REQUIREMENTS

- Input field for stock symbol
- Display area for stock data
- Chart section for visualization
- Clean and simple layout

9. TESTING REQUIREMENTS

- Test with valid stock symbols
- Test with invalid inputs
- Verify chart accuracy
- Check system performance

10. EXPECTED OUTPUT

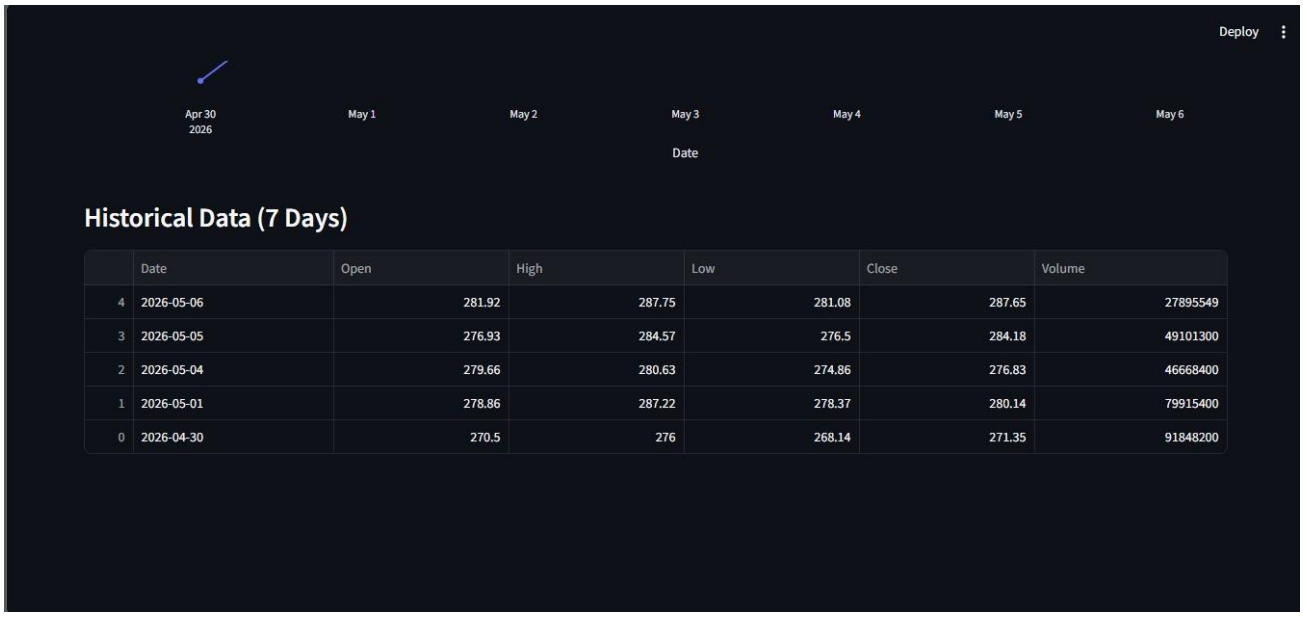
- Display of stock price information
- Graph showing stock trends
- Simple and interactive web interface

The following screenshots show the complete expected output of the system across all test cases, including valid symbols and invalid input handling:

Test Case 1 — AAPL (7 Days): Valid Symbol

AAPL price trend line chart (Apr 30 – May 6, 2026) with markers on each data point, and the Historical Data table below it showing 5 trading days of OHLCV values — demonstrating successful chart generation and data display for a valid symbol over 7 days.

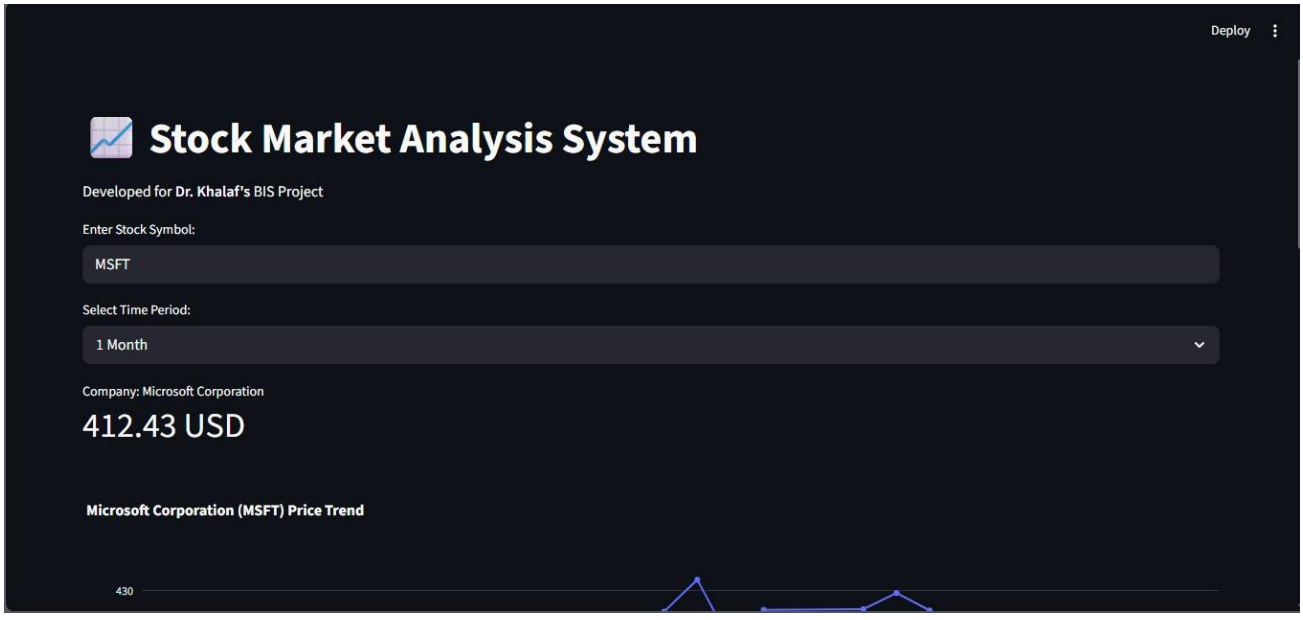
The main application interface showing the stock symbol input field (AAPL) and time period dropdown (7 Days). Displays the company name and current price as a metric card — demonstrating the clean, simple layout requirement.



	Date	Open	High	Low	Close	Volume
4	2026-05-06	281.92	287.75	281.08	287.65	27895549
3	2026-05-05	276.93	284.57	276.5	284.18	49101300
2	2026-05-04	279.66	280.63	274.86	276.83	46668400
1	2026-05-01	278.86	287.22	278.37	280.14	79915400
0	2026-04-30	270.5	276	268.14	271.35	91848200

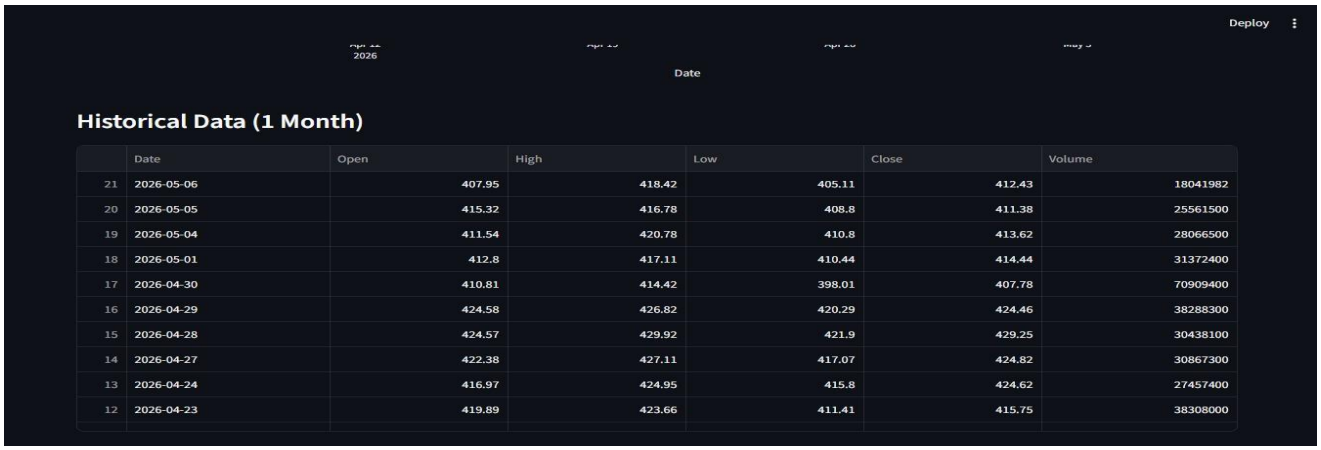
MSFT price trend chart spanning April to May 2026 (1-month period), showing a clear upward trend from ~370 USD to ~430 USD range with interactive markers — confirming that dynamic chart updates based on user time period selection work correctly.

Test Case 2 — MSFT (1 Month): Valid Symbol



The same UI with MSFT entered and 1 Month selected, confirming the input field and selector work correctly for different symbols and time periods

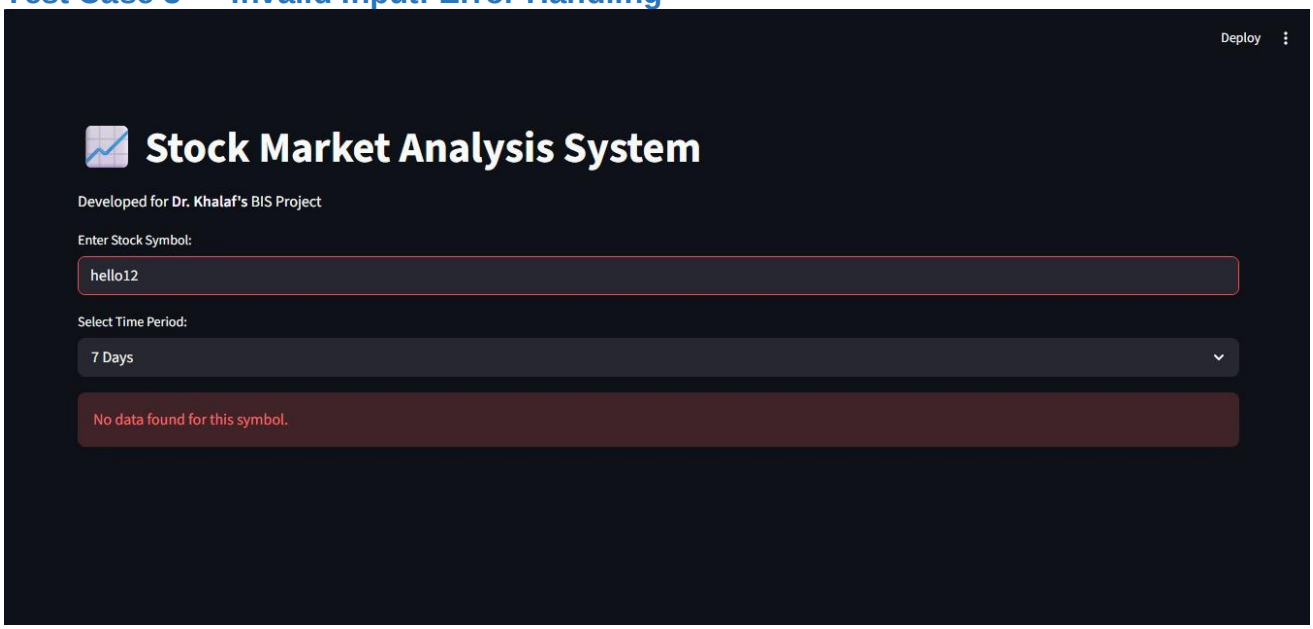
MSFT Historical Data table (1 Month) listing 21 trading days of OHLCV data sorted by date descending — confirming that 1-month historical data is correctly fetched, cleaned, and displayed..



	Date	Open	High	Low	Close	Volume
21	2026-05-06	407.95	418.42	405.11	412.43	18041982
20	2026-05-05	415.32	416.78	408.8	411.38	25561500
19	2026-05-04	411.54	420.78	410.8	413.62	28066500
18	2026-05-01	412.8	417.11	410.44	414.44	31372400
17	2026-04-30	410.81	414.42	398.01	407.78	70909400
16	2026-04-29	424.58	426.82	420.29	424.46	38288300
15	2026-04-28	424.57	429.92	421.9	429.25	30438100
14	2026-04-27	422.38	427.11	417.07	424.82	30867300
13	2026-04-24	416.97	424.95	415.8	424.62	27457400
12	2026-04-23	419.89	423.66	411.41	415.75	38308000

MSFT price trend chart spanning April to May 2026 (1-month period), showing a clear upward trend from ~370 USD to ~430 USD range with interactive markers — confirming that dynamic chart updates based on user time period selection work correctly.

Test Case 3 — Invalid Input: Error Handling



Error handling test with the invalid symbol "hello12". The application displays a red error message: "No data found for this symbol." without crashing — confirming the invalid input handling requirement is fully implemented.

11. CONCLUSION

This project demonstrates basic knowledge of stock market data analysis and web application development using Streamlit. It is suitable for beginners and provides a foundation for more advanced systems.

12. TEAM TASK ALLOCATION

Shahd Maher
Member 1 – Project Lead / Backend Developer

Main Role: API Integration + Core Logic

- Set up Python project structure
- Integrate stock data API using yFinance
- Handle stock symbol input and validation
- Fetch current + historical stock data
- Manage error handling (invalid symbols, API failures)

Zeina Sherif
Member 2 – Data Processing & Analysis

Main Role: Data Handling

- Clean and structure stock data using Pandas
- Extract required fields (open, close, high, low, volume)
- Prepare data for visualization
- Compute simple indicators (optional): moving average, daily change

Rama said
Member 3 – Visualization Developer

Main Role: Charts & Graphs

- Create stock trend charts using Matplotlib or Plotly
- Build interactive visualizations (line charts, trends)
- Design graph layout for clarity
- Ensure charts update dynamically based on user input

Mayar Sherief
Member 4 – UI Developer / Tester / Documentation

Main Role: Streamlit UI + Testing

- Build Streamlit interface (input box, buttons, layout)
- Display stock info and charts in UI
- Improve user experience (clean design)
- Test application with valid/invalid inputs
- Prepare documentation (report, screenshots, PPT if needed)

Rayan Hesham
Member 5 – Supporting Developer

Main Role: Integration & Quality Assurance

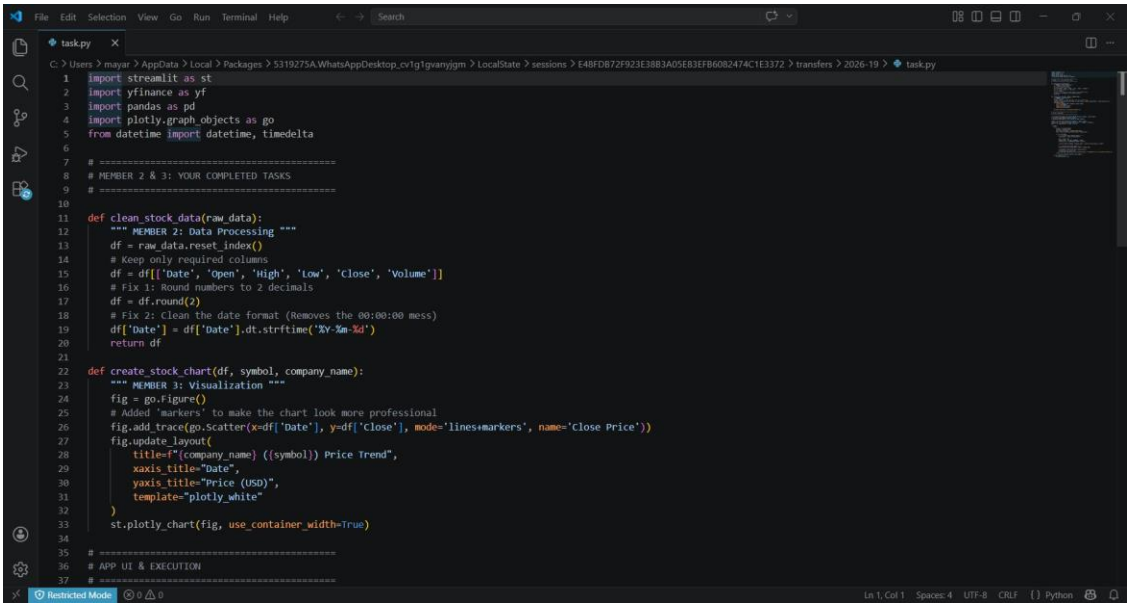
- Assist with integrating all team members' code into the final application
- Review and test the complete system end-to-end
- Support debugging and code consistency across modules
- Contribute to final documentation and report preparation

13. SOURCE CODE

The following screenshots present the complete source code of the application (task.py), written in Python. The code is organized into two main functional blocks corresponding to the team task allocation.

13.1 Data Processing & Visualization Functions (Lines 1–37)

This block contains the project imports and two core functions: clean_stock_data() which resets the dataframe index, selects the required OHLCV columns, rounds all numerical values to 2 decimal places, and strips the timestamp from the date field; and create_stock_chart() which builds an interactive Plotly scatter chart with lines and markers, configures axis titles, applies the plotly_white theme, and renders it in Streamlit.

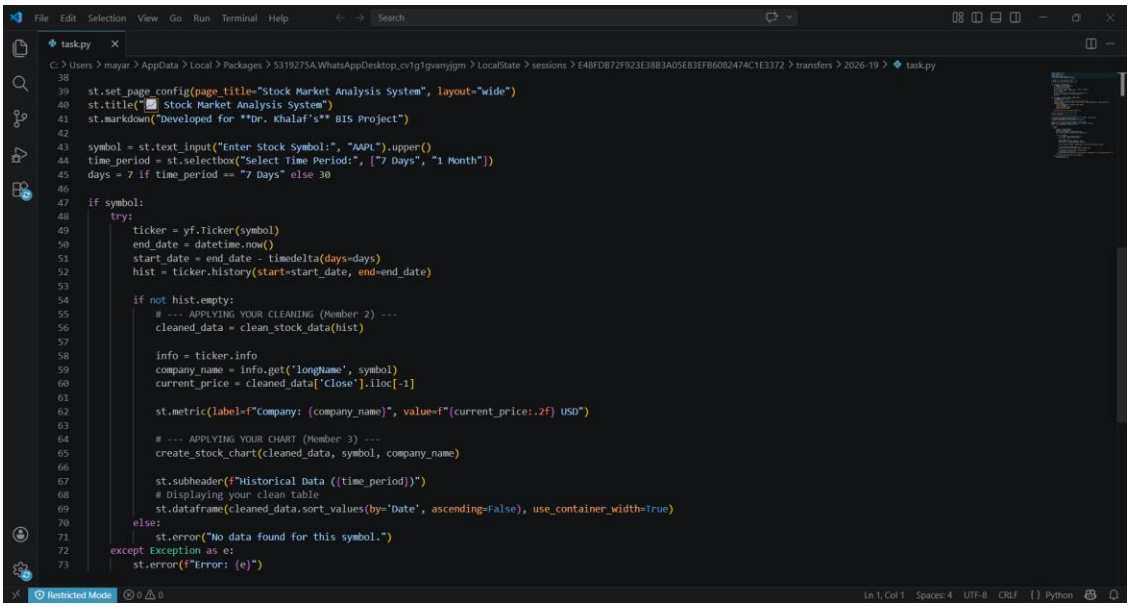


```python
import streamlit as st
import yfinance as yf
import pandas as pd
import plotly.graph_objects as go
from datetime import datetime, timedelta

# ==========================================
# MEMBER 2 & 3: YOUR COMPLETED TASKS
# ==========================================

def clean_stock_data(raw_data):
    """ MEMBER 2: Data Processing """
    df = raw_data.reset_index()
    # Keep only required columns
    df = df[['Date', 'Open', 'High', 'Low', 'Close', 'Volume']]
    # Fix 1: Round numbers to 2 decimals
    df = df.round(2)
    # Fix 2: Clean the date format (Removes the 00:00:00 mess)
    df['Date'] = df['Date'].dt.strftime('%Y-%m-%d')
    return df

def create_stock_chart(df, symbol, company_name):
    """ MEMBER 3: Visualization """
    fig = go.Figure()
    # Added 'markers' to make the chart look more professional
    fig.add_trace(go.Scatter(x=df['Date'], y=df['Close'], mode='lines+markers', name='Close Price'))
    fig.update_layout(
        title=f"{company_name} ({symbol}) Price Trend",
        xaxis_title="Date",
        yaxis_title="Price (USD)",
        template="plotly_white"
    )
    st.plotly_chart(fig, use_container_width=True)

# ==========================================
# APP UI & EXECUTION
# ==========================================
```

task.py — Lines 1–37: Library imports (streamlit, yfinance, pandas, plotly, datetime), the clean_stock_data() data processing function , and the create_stock_chart() visualization function .

13.2 App UI & Execution Logic (Lines 38–73)

This block handles the Streamlit UI setup: page configuration, app title, markdown subtitle, stock symbol text input defaulting to AAPL, and a time period selectbox with 7 Days / 1 Month options. It then contains the main execution logic: computing the date range, calling yFinance to fetch historical data, applying the cleaning and chart functions, displaying the company name and current price as a metric, rendering the chart, showing the sorted historical data table, and handling both empty data and exception errors.



```python
st.set_page_config(page_title="Stock Market Analysis System", layout="wide")
st.title("📈 Stock Market Analysis System")
st.markdown("Developed for **Dr. Khalaf's** BIS Project")

symbol = st.text_input("Enter Stock Symbol:", "AAPL").upper()
time_period = st.selectbox("Select Time Period:", ["7 Days", "1 Month"])
days = 7 if time_period == "7 Days" else 30

if symbol:
    try:
        ticker = yf.Ticker(symbol)
        end_date = datetime.now()
        start_date = end_date - timedelta(days=days)
        hist = ticker.history(start=start_date, end=end_date)

        if not hist.empty:
            # --- APPLYING YOUR CLEANING (Member 2) ---
            cleaned_data = clean_stock_data(hist)

            info = ticker.info
            company_name = info.get('longName', symbol)
            current_price = cleaned_data['Close'].iloc[-1]

            st.metric(label=f"Company: {company_name}", value=f"{current_price:.2f} USD")

            # --- APPLYING YOUR CHART (Member 3) ---
            create_stock_chart(cleaned_data, symbol, company_name)

            st.subheader(f"Historical Data ({time_period})")
            # Displaying your clean table
            st.dataframe(cleaned_data.sort_values(by='Date', ascending=False), use_container_width=True)
        else:
            st.error("No data found for this symbol.")
    except Exception as e:
        st.error(f"Error: {e}")
```

task.py — Lines 38–73: Streamlit page config, UI controls (stock input, time period selector), yFinance API call, data display logic, chart rendering, historical data table, and error handling blocks.